Student Task Tracker

Programming in Java-CSE2006

Project Report

Student Name	Registration Number
Kaustubh Upadhyay	24BAI10060
Slot	**Date of Submission**
A21+A22	30th-March-2026

1. Introduction

This report explains the detail working and development of **Student Task Tracker,** a java based Desktop application made to handle the issue of academic task management. By using the Object Oriented Programming(OOP) principle, Java Collections and Swing based GUI, this project provide a user centred soltion to organize, track and complete assignments. The project gives an emphasis on code modularity, error management and data persistence.

2. Problem Statement

1. The Problem

Now a days in higher education, students use a lot of Learning Management Systems (LMS) like Moodle, Canva etc. These platforms host content but they often fail to provide a personalised, cross subject To-Do experience.

2. Need

Brain Load(cognitive load): Manually tracking many deadlines in various subjects increases mental fatigue.

Priority: Without a sorted view, student often work on “loud” tasks rather than urgent ones.

Persistence: Simple script lose data on exit. A tool in this work should remember user’s inputs.

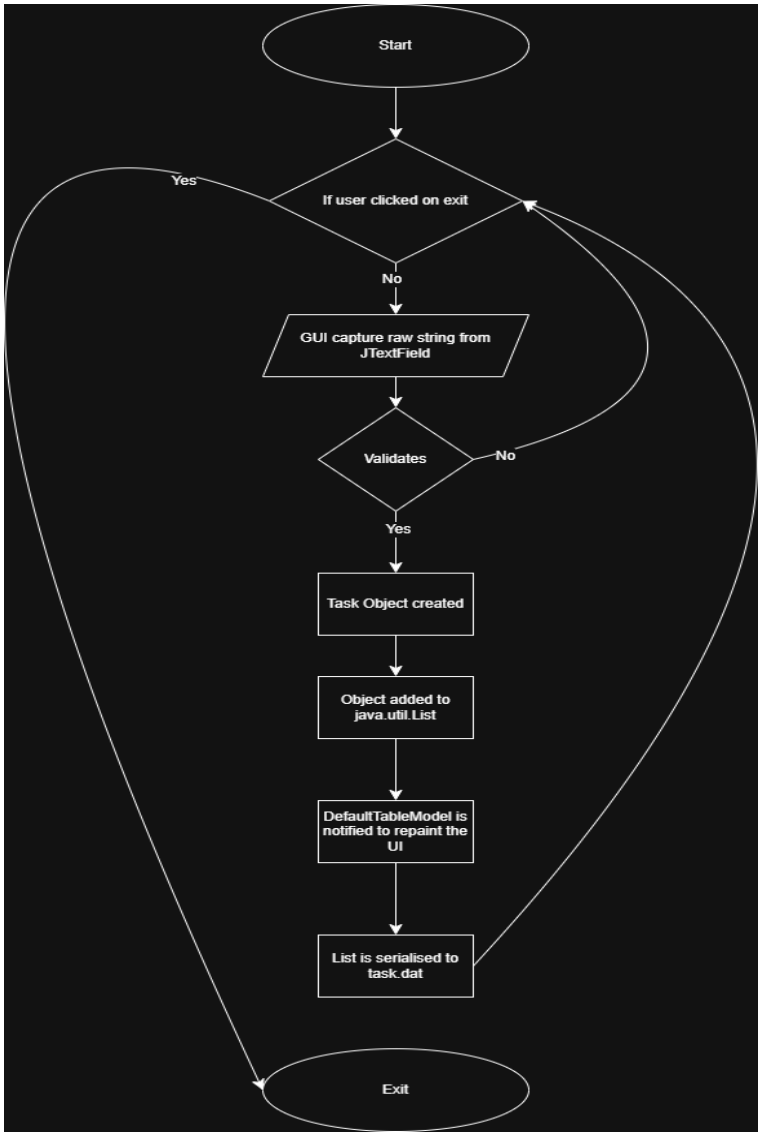
3. Architecture

1. The MVC (Model View Control) Pattern

This application was designed in a MVC architectural pattern to ensure separation of concerns:

1. **Model(Task.java):** Manages the data and logic of what is task.
2. **View(TaskMang.java):** Defines the visual Layout and user feedback.
3. **Controller(TaskMang.java):** Accts as bridge, processing user input and updating the Model and View.

2. Data Flow Diagram



When a user adds a task:

1. The GUI captures raw strings from JTextFields.
2. The controller validates the format.
3. A task object is initialised.
4. The object is added to java.util.List.
5. The DefaultTableModel is notified to repaint the UI.
6. The list is serialised to tasks.dat.

4. Implementation

1. *Object Oriented Principles*

Encapsulation

The Task class encapsulates four attributes: title, subject, dueDate and isCpmpleted. By making these private we prevent illegal state errors, i.e., task cannot be marked done without using markAsCompleted() method.

Serialization

To achieve persistence, I implemented the java.io.Serializable interface. This is maker interface that tells the JVM that this object can be converted into a flat byte stream.

2. *Java Collection Framework*

ArrayList<Taks> over a standard Task[] array because:

Dynamic Growth: Arrays in Java are fixed size. Since workload is dynamic, ArrayList's ability to grow by 50% of its capacity when full was essential.

Iterative efficiency: Using enhanced for-loop allows clean rendering of table data.

3. *GUI Construction*

The interface uses a nested layout strategy:

1. **BorderLayout:** The Outer Shell. The input dorm stays at the NORTH, table at CENTER and actions at the SOUTH.

2. **GridLayout:** Ensures that the input labels and text fields stay perfectly aligned in a 2×4 matrix.

5. Code

Task.java

```
import java.io.Serializable;
import java.time.LocalDate;
public class Task implements Serializable
 {
   private String title;
   private String subject;
   private LocalDate dueDate;
   private boolean isCompleted;
   public Task(String title, String subject,LocalDate dueDate)
   {
     this.title=title;
     this.subject=subject;
     this.dueDate=dueDate;
     this.isCompleted=false;
   }
   public String getTitle()
   {
     return title;
   }
   public String getSubject()
   {
     return subject;
   }
   public LocalDate getDueDate()
   {
     return dueDate;
   }
   public boolean isCompleted()
   {
     return isCompleted;
   }
   public void markAsCompleted()
     {
```

```
    this.isCompleted=true;
    }
  @Override
  public String toString()
  {
    String status=isCompleted ? "[Done]" : "[Pending]";
    return String.format("%s %s |Subject: %s | Due: %s", status, title,
subject, dueDate);
  }
}
```

TaskMang.java

```
import java.util.List;
import java.util.ArrayList;
import java.io.*;
import java.time.LocalDate;
import javax.swing.*;
import javax.swing.table.DefaultTableModel;
import java.awt.*;
public class TaskMang extends JFrame
{
 private static String
FILE_NAME="C:\\Users\\kstbh\\OneDrive\\Desktop\\tasks.dat";
 private static java.util.List<Task> tasks;
 private DefaultTableModel tableModel;
 private JTable taskTable;
 private JTextField titleField, subjectField, dateField;

 public TaskMang()
 {
  tasks=loadTasks();
  initializeUI();
 }
 private void initializeUI()
 {
  setTitle("Student Task Tracker");
  setSize(700,500);
  setDefaultCloseOperation(EXIT_ON_CLOSE);
  setLayout(new BorderLayout(10,10));
  JPanel input=new JPanel(new GridLayout(2,4,5,5));
```

```
  input.setBorder(BorderFactory.createEmptyBorder(10,10,10,10));
  titleField=new JTextField();
  subjectField=new JTextField();
  dateField=new JTextField(LocalDate.now().toString());
  input.add(new JLabel("Task Title:"));
  input.add(new JLabel("Subject:"));
  input.add(new JLabel("Due Date(YYYY-MM-DD):"));
  input.add(new JLabel(""));
  input.add(titleField);
  input.add(subjectField);
  input.add(dateField);
  JButton addButton=new JButton("Add Task");
  addButton.addActionListener(e->addTask());
  input.add(addButton);

  String[] col={"Status","Title","Subject","Due Date"};
  tableModel= new DefaultTableModel(col,0);
  taskTable=new JTable(tableModel);
  refreshTable();

  JPanel actionPanel=new JPanel();
  JButton completeButton=new JButton("Mark Selected as Complete");
  JButton saveButton=new JButton("Save and Exit");
  completeButton.addActionListener(e->markComplete());
  saveButton.addActionListener(e->{saveTasks(); System.exit(0);});
  actionPanel.add(completeButton);
  actionPanel.add(saveButton);
  add(input, BorderLayout.NORTH);
  add(new JScrollPane(taskTable), BorderLayout.CENTER);
  add(actionPanel, BorderLayout.SOUTH);
  setLocationRelativeTo(null);
}
private void addTask()
{
  try{
    String title=titleField.getText();
    String subject=subjectField.getText();
    LocalDate date=LocalDate.parse(dateField.getText());
    if(title.isEmpty()||subject.isEmpty())
```

```
      {
        throw new Exception();
      }
      tasks.add(new Task(title, subject, date));
      refreshTable();
      clearFields();
    }catch(Exception e)
      {
        JOptionPane.showMessageDialog(this,"Please enetr valid details
and YYYY-MM-DD format.");
      }
  }

  private void markComplete()
  {
    int selectedRow=taskTable.getSelectedRow();
    if(selectedRow!=-1)
    {
      tasks.get(selectedRow).markAsCompleted();
      refreshTable();
    }
    else
    {
      JOptionPane.showMessageDialog(this, "Please select a task from
the table first.");
    }
  }
  private void refreshTable()
  {
    tableModel.setRowCount(0);
    for(Task t:tasks)
      {
        Object[] row={
          t.isCompleted()?"[Done]":"[Pending]",
          t.getTitle(),
          t.getSubject(),
          t.getDueDate(),
        };
        tableModel.addRow(row);
      }
```

```
  }
  private void clearFields()
  {
    titleField.setText("");
    subjectField.setText("");
    dateField.setText(LocalDate.now().toString());
  }
  private void saveTasks()
  {
    try(ObjectOutputStream oos=new ObjectOutputStream(new
FileOutputStream(FILE_NAME)))
      {
        oos.writeObject(tasks);
      }catch (IOException e)
      {
        JOptionPane.showMessageDialog(this, "Error saving data");
      }
  }
  @SuppressWarnings("unchecked")
  private List<Task> loadTasks()
  {
    File file=new File(FILE_NAME);
    if(!file.exists()) return new ArrayList<>();
    try(ObjectInputStream ois= new ObjectInputStream(new
FileInputStream(FILE_NAME)))
      {
        return(List<Task>) ois.readObject();
      }catch(Exception e)
      {
        return new ArrayList<>();
      }
  }
  public static void main(String[] args)
  {
    SwingUtilities.invokeLater(()->new TaskMang().setVisible(true));
  }
}
```

6. Challenges, Debugging and Resolution

1. Ambiguous List conflict

Challenge: On importing java.awt.* and java.util.*, the compiler threw error: reference to List is ambiguous.

Resolution: This occurred because both the packages contain a List class. This was resolved using the full name java.util.List for the data structure, thus utilising namespace management.

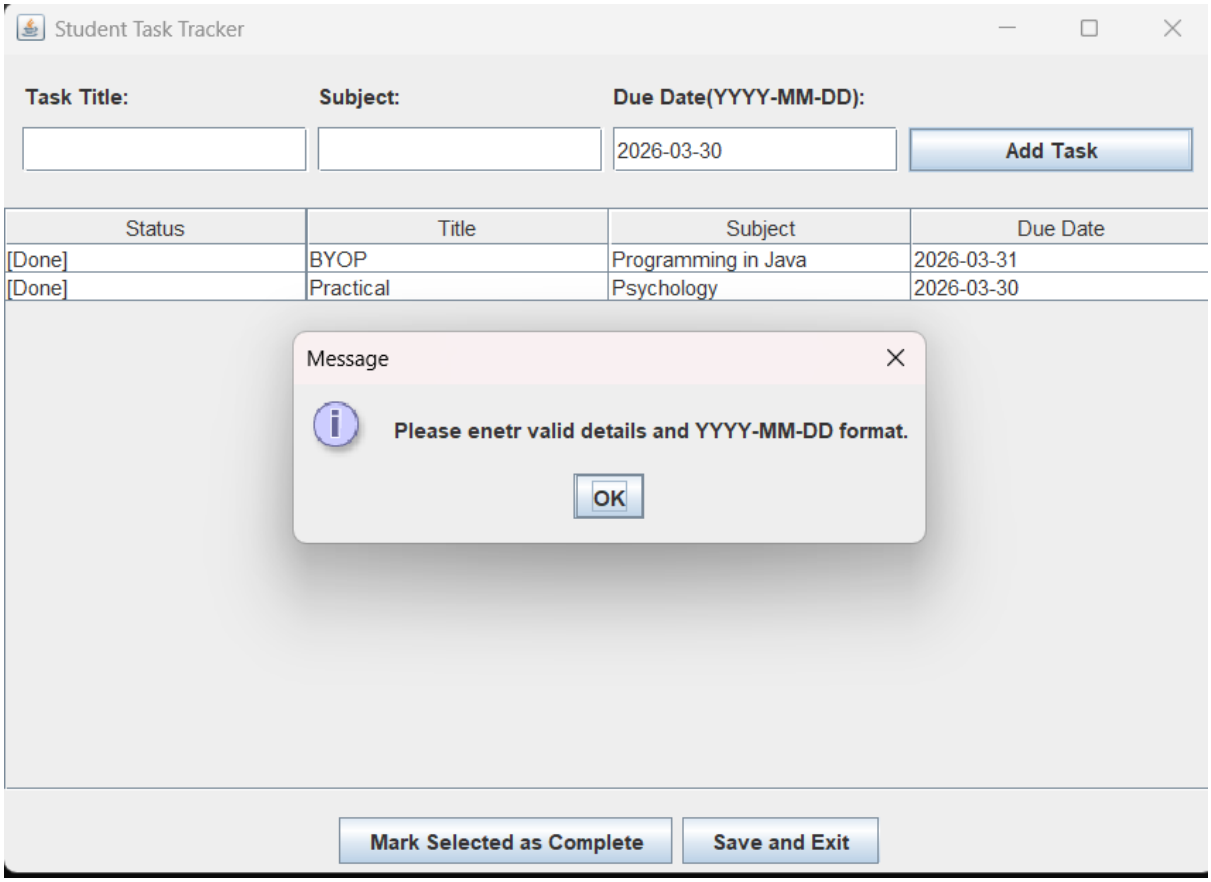
2. Date Parsing Logic

Challenge: The user often enter date incorrectly.

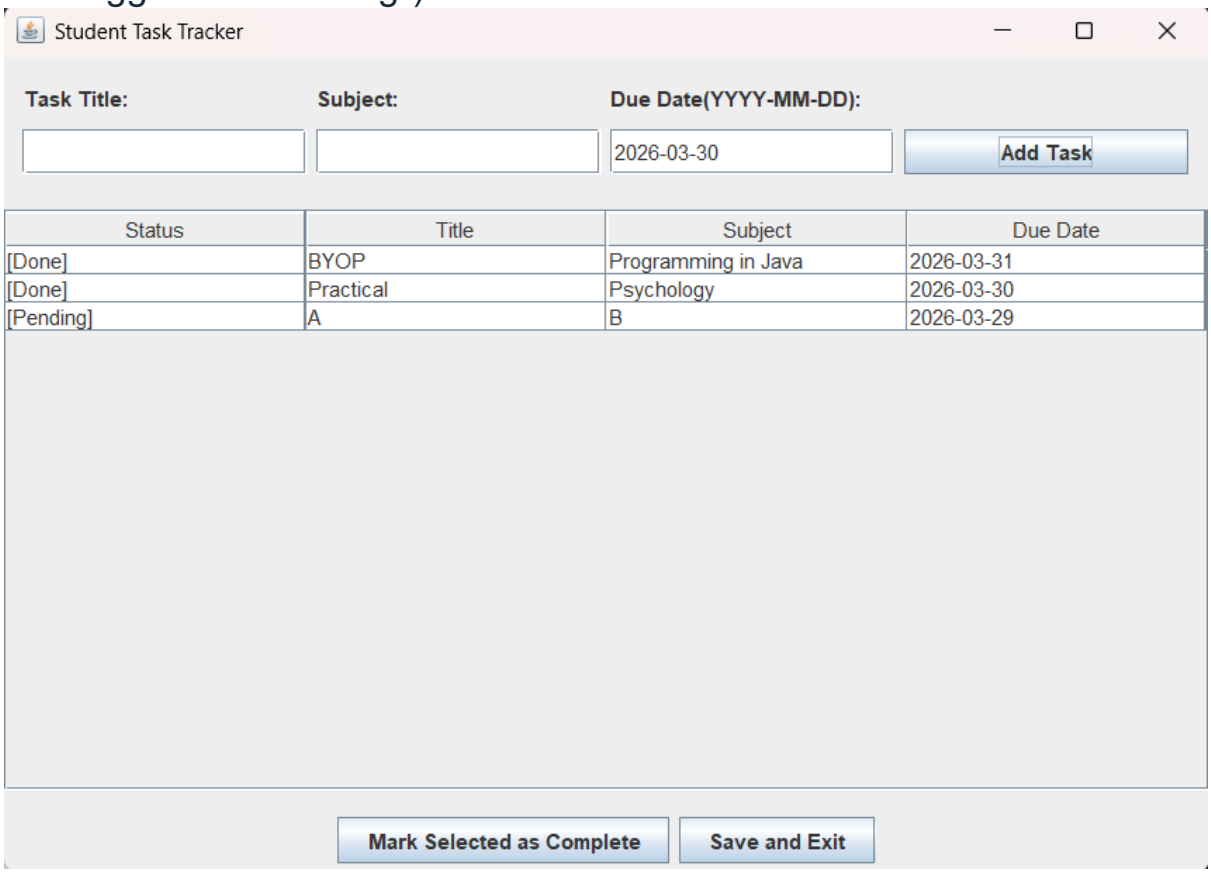
Resolution: Implemented java.time.LocalDate.parse(). If format is incorrect, DateTimeParseException will show up and JOptionPane alter will guide the user to enter correct format.

7. Testing(Manual)

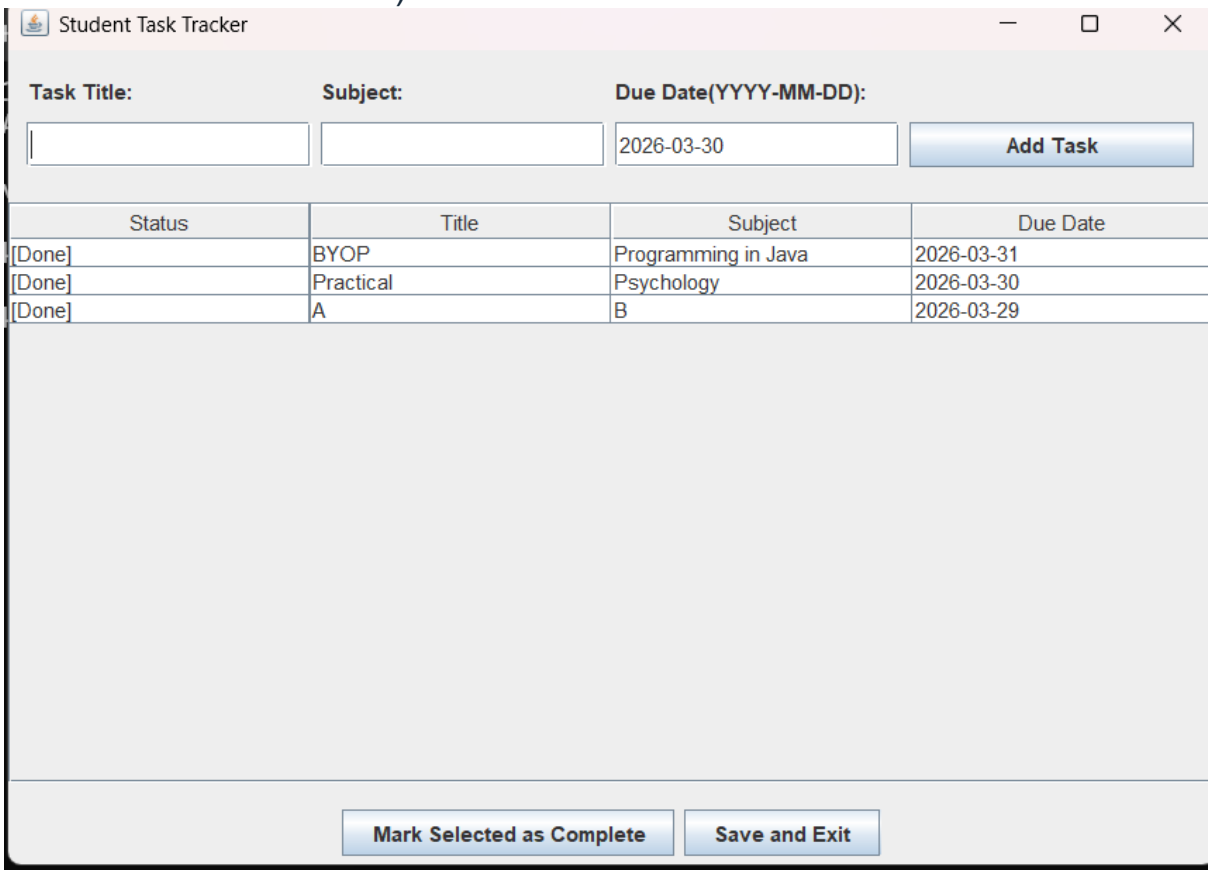
- **Test Case 1:** Add a task with an empty title. (Result: Prevented by validation logic).



- **Test Case 2:** Add a task with a date in the past. (Result: Accepted, but flagged as "Pending").



- **Test Case 3:** Close app and restart. (Result: Data successfully reloaded from tasks.dat).



8. Future

1. Database Integration: Changing from file based storage to MySQL using JDBC.
2. Notification System: Trigger system alerts using background threads.
3. Priority Level: Colour coded system for showing priority of the tasks.

9. Conclusion

This project demonstrates the power of Java as a tool for solving real world problems, Through applications of OOP, Exception Handling and GUI development. A tool is created that is practically useful.

10. References

1. Oracle Java Documentation
2. Java Swing Tutorials
3. VITyarthi cource: Programming in Java